

Group symbol: **2**

Team: **1**

Project title: **ListItUp**

Team members (*filled by PM, Team Leader*):

No	Name	Surname	Student ID	Role
1	Carmelo Aidan	Romero Pérez	293936	<i>PM, Team Leader</i>
2	Álvaro	Puebla Ruisánchez	293867	<i>Team member</i>
3	Carmen	Gutiérrez Sánchez	293906	<i>Team member</i>
4	David	Sosa Domínguez	293885	<i>Team member</i>
5	Diego	García Agrelo	293939	<i>Team member</i>
6	Miryam	Merchán León	293907	<i>Team member</i>

1. Elaboration of application concept (S1)

1.1. Project (business) goals

Present as a tree of the project's objectives, i.e., decomposition of general project objective into specific ones. A goal should be:

- relevant to the project,
- important from the perspective of the end-user,
- achievable,
- measurable,
- realistic (achievable in terms of constraints: cost and time-oriented, technological, organizational, human resources, legal).

Specific Business Goals:

- **Make Content Creation Effortless**
 - **Description:** We want the platform to be so simple to use that anyone can build and share a list right away. By removing any confusing steps, we aim to grow our collection of user-made lists as fast as possible.
 - **Measurable Target:** At least 80% of our test users can successfully create and publish a list on their own, without asking for help.
- **Provide Instant Value Through Discovery**
 - **Description:** We want to grab users' attention the moment they open the app. The goal is to make sure they find interesting lists immediately, so they stay on the platform longer.
 - **Measurable Target:** A new user should be able to find a relevant and interesting list within 10 to 15 seconds of opening the application.
- **Grow an Active Community**
 - **Description:** We want to turn the app into a lively social space. The goal is to encourage users to interact with each other's content, rather than just reading it and leaving.
 - **Measurable Target:** Achieve at least one interaction (such as a like, save, or comment) from 15% of the users testing our app during the evaluation phase.
- **Deliver a Fast and Reliable Experience**
 - **Description:** We want the website to be smooth and stable. If the platform is fast and doesn't crash, users will trust it and keep coming back on both mobile and desktop devices.
 - **Measurable Target:** Keep the system running and available at least 99.5% of the time during the pilot period and make sure page load times stay under 2 to 3 seconds.

1.2. Identification of project's internal and external Stakeholders

Identify the role of each **internal** stakeholder in the project, along with a textual description.

Symbol	Name	Role	Description
I1	Carmelo A. Romero	Cross-functional team member	Leads the team, coordinates stakeholders, owns schedule and scope, primary contact for sponsor/IT; also contributes across project tasks as needed.
I2	Álvaro Puebla	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I3	David Sosa	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I4	Carmen Gutiérrez Sánchez	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I5	Diego García	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.
I6	Miryam Merchán León	Cross-functional team member	Contributes across planning, development, testing, deployment and documentation as required.

Identify the role of each **external** stakeholder in the project, along with a textual description.

Symbol	Name	Role	Description
E1	Casual Users	Standard Users	Everyday users who use the app to discover cool recommendations and share their own interests. They can browse, create, edit, and share their own lists. They interact with the community by leaving likes, comments, and saving other people's lists to their personal collections.
E2	Verified Creators	Content Creators/Influencers	High-profile users, influencers, or experts who have a "verified" status on the platform. Alongside standard features, they have access to exclusive tools: a "Creator Dashboard" to view profile and list analytics (like views, saves, and link clicks), a verified badge on their profile, and the ability to pin top lists to their page.
E3	App Administrators	Platform Moderators	A special user role (managed by our team during this phase) with permissions to manage the platform. They can delete inappropriate test lists, remove buggy comments, and keep the database clean during the testing period.
E4	Third-Party API Providers	Technology Partners	Providers of external APIs integrated into the platform: social login (Google, GitHub), image hosting, and link-metadata enrichment. They define technical constraints and service-level agreements.

1.3. Domain description

- Description of phenomena in the application domain that can be observed.
- Definitions of concepts that abstract these phenomena.
- The concepts include:
 - entities - things that one can “distinguish” from the fraction of reality,
 - relationships - either properties (unary relationships of entities) that yield some information about them or relationships between two or more entities (binary or n -ary relations),
 - events - things that can happen (occur).
- Use UML class diagram to depict entities (class with attributes), relationships (associations).
- This diagram **must** be supplemented with the list of entities, relationships and events along with their description in natural language.

Domain Phenomena:

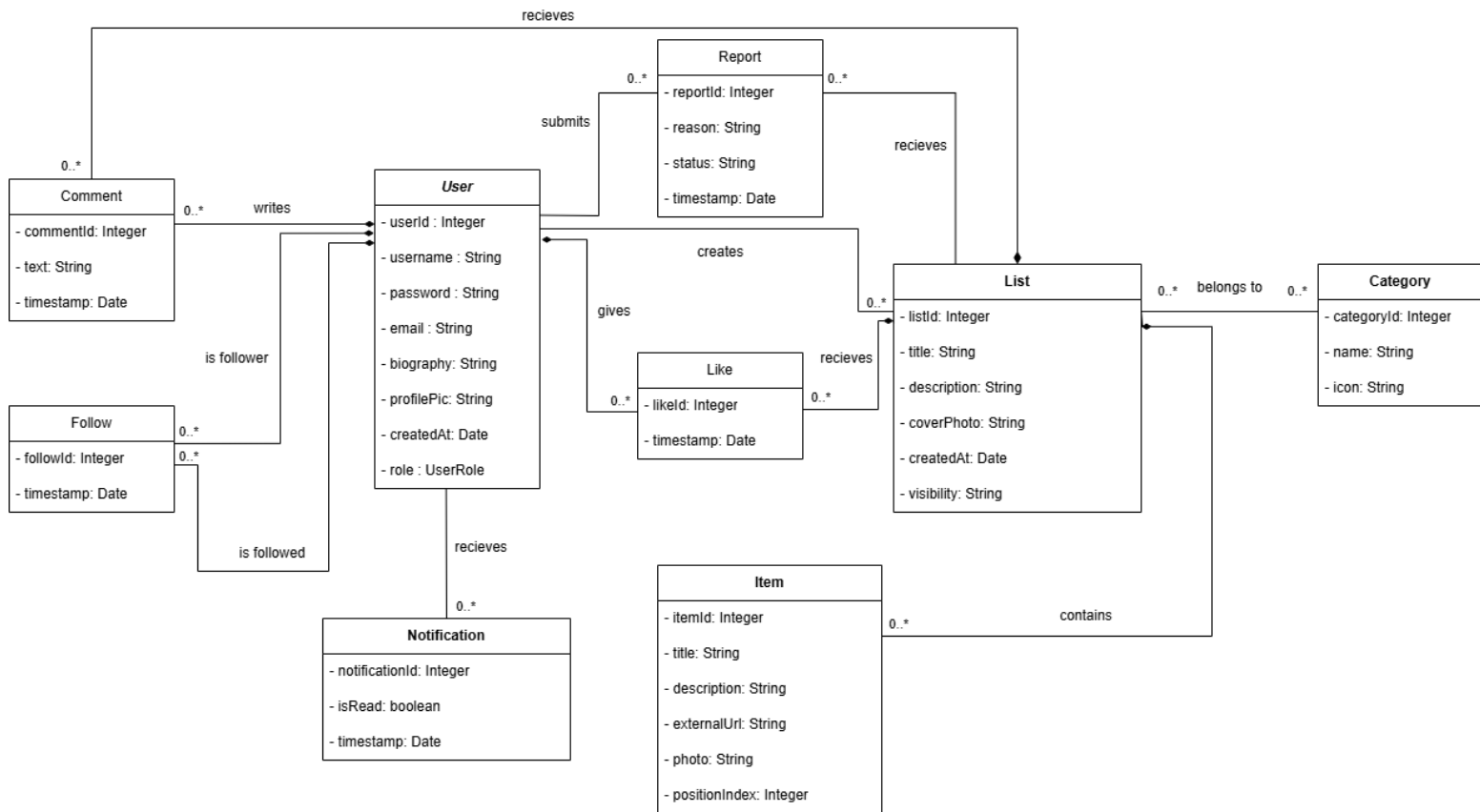
- **User Registration & Roles:** Users sign up to the platform. They can be standard users, "verified" creators (influencers/VIPs), or admins who manage the platform.
- **Creating and Managing Lists:** Users build collections of their favorite things (e.g., top 10 movies, best study tools). A list can be public or private, and the creator can edit or update it over time.
- **Adding Items:** Inside each list, users add specific items. Each item usually has a title, a short description, an image, and an external link to the product or place.
- **Social Interactions and Discovery:** Users browse the app's feed or search by categories to discover new lists. If they find an interesting list, they can leave a "Like", write a comment, or save the list to their personal profile to check it out later.
- **Follower System and Creator Stats:** Users can follow their favorite creators to see their new lists on their home feed. Verified creators get access to extra data, like how many views or saves their lists are getting.
- **Follow graph:** Users follow curators, creating a directed social graph. Followers receive a personalized feed of newly published or updated lists from those they follow.
- **Alerts and Notifications:** The app sends a simple notification when someone likes a user's list, leaves a comment, or when a followed creator posts something new. Also, if there is an update to a saved list, trigger notifications to relevant users via in-app and/or email channels.
- **Content moderation:** Users can report lists, items, or comments that violate platform guidelines. Reported content enters a moderation review queue managed by administrators.

Definition of Concepts:

1. **User** - A registered account on the platform.
 - *Key attributes:* (userId, username, email, password, biography, profilePicture, role, createdAt, |standard, verified, admin|)ç
 - *Relationships:*
 - **Creates (1-to-Many):** One (1) User can create many (0..*) Lists.
 - **Follows (Many-to-Many):** A User can be a follower (0..) of other Users, and be followed by many (0..) other Users.

- **Writes (1-to-Many):** One (1) User can write many (0..*) Comments.
 - **Gives (1-to-Many):** One (1) User can give many (0..*) Likes.
 - **Submits (1-to-Many):** One (1) User can submit many (0..*) Reports.
 - **Receives (1-to-Many):** One (1) User can receive many (0..*) Notifications.
2. **List** - A list of items made by an user.
- *Key attributes:* (listId, title, description, coverPhoto, createdAt, visibility | public, private|)
 - *Relationships:*
 - **Created by (Many-to-1):** Many (0..*) Lists belong to exactly one (1) User.
 - **Contains (1-to-Many):** One (1) List contains many (0..*) Items.
 - **Belongs to Category (Many-to-1):** Many (0..*) Lists can be grouped under one (1) Category.
 - **Receives (1-to-Many):** One (1) List can receive many (0..) *Comments and many (0..) Likes.*
3. **Item** - A single entry within a List.
- *Key attributes:* (itemId, title, description, externalUrl, photo, positionIndex,)
 - *Relationships:*
 - **Belongs to (Many-to-1):** Many (0..*) Items belong to exactly one (1) List.
4. **Category** - To group lists and find them on the feed.
- *Key attributes:* (categoryId, name, icon)
 - *Relationships:*
 - **Contains (1-to-Many):** One (1) Category contains many (0..*) Lists.
5. **Comment** - A textual contribution by a user on a CuratedList.
- *Key attributes:* (commentId, text, time_stamp)
 - *Relationships:*
 - **Written by (Many-to-1):** Many (0..*) Comments are written by exactly one (1) User.
 - **Belongs to (Many-to-1):** Many (0..*) Comments belong to exactly one (1) List.
6. **Notification** - A system message generated by a domain event.
- *Key attributes:* (notificationId, message, is_read, timestamp)
 - *Relationships:*
 - **Received by (Many-to-1):** Many (0..*) Notifications are received by exactly one (1) User.
7. **Report** - a user-submitted content moderation request.
- *Key attributes:* (report_id, reason, status | open, reviewed, resolved|, timestamp).
 - *Relationships:*
 - **Submitted by (Many-to-1):** Many (0..*) Reports are submitted by exactly one (1) User.

SSD 2026 - Project Report S1



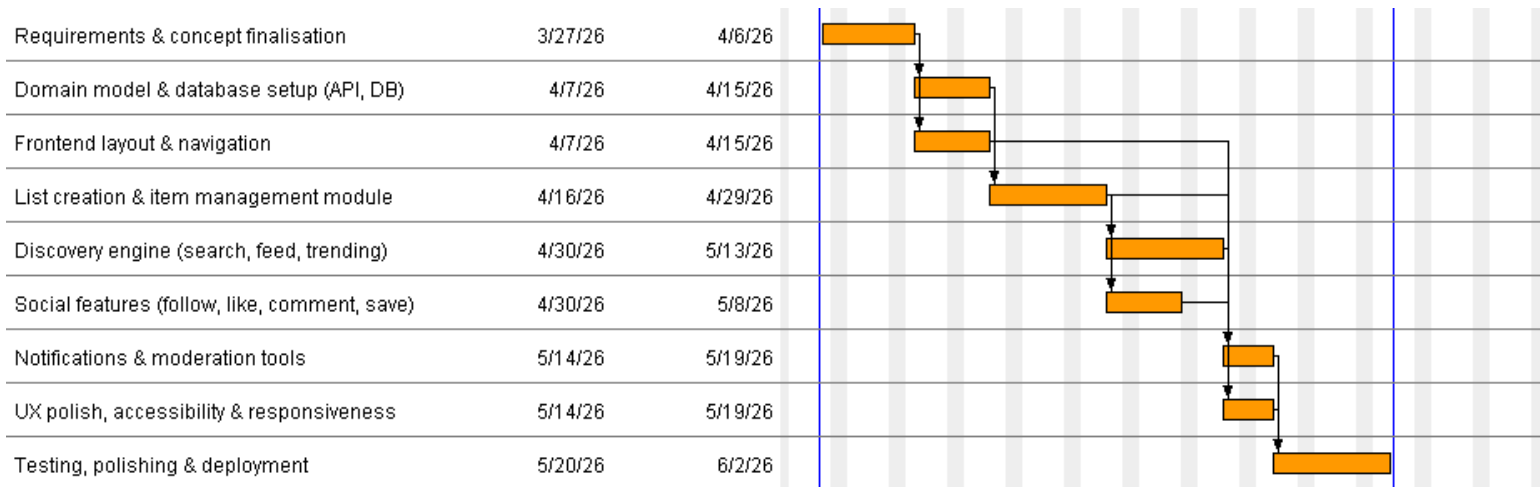
Class Diagram

1.4. Project schedule (Gantt chart)

- tasks with dependencies,
- start time and duration,
- resource allocation: staff, costs, equipment.

Overview: 7-week MVP. Tasks include dependencies, start week (W1 = project start), duration in weeks, primary assigned role, and estimated staff-hours.

ID	Task	Duration	Depends on	Notes
T1	Requirements & concept finalisation	1 wk	-	Define list types and workflows
T2	Domain model & database setup (API, DB)	1 wk	T1	Entities, relations, policies
T3	Frontend layout & navigation	1 wk	T1	React base, pages, component library
T4	List creation & item management module	1.5 wk	T2	Curator editor, draft/publish flow
T5	Discovery engine (search, feed, trending)	1.5 wk	T4	Full-text search, trending, personalised feed
T6	Social features (follow, like, comment, save)	1 wk	T4	Follow graph, interactions, saved collections
T7	Notifications & moderation tools	0.5 wk	T5, T6	Follow graph, interactions, saved collections
T8	UX polish, accessibility & responsiveness	0.5 wk	T3-T6	WCAG 2.1 AA pass, mobile review
T9	Testing, polishing & deployment	1.5 wk	T7, T8	QA + Docker deploy



Gantt Diagram

Team Effort & Budget

- **Team effort (approx.):** We estimate this MVP will take approximately 384 hours of total work distributed across our team of six.
- **Estimated theoretical cost (approx.):** If this were a real-world commercial project, paying developers and setting up enterprise infrastructure for this 7-week MVP would cost roughly between €7,000 and €10,000.

- **Actual project budget:** 0€. Since we are students, we are not actually spending any money on this project.
- **Resources:** To make this 0€ budget work, we are doing all the coding on our own shared personal developer laptops. For the infrastructure, we are strictly relying on the completely free tiers of cloud databases and hosting services. We are also using free-tier third-party APIs to handle complex features like OAuth user login, image hosting, and sending automated emails..

1.5. Identification of existing or alternative solutions

Each competitor should be characterized with

- name of the project/application,
- main features,
- advantages,
- disadvantages.

Are.na

- **Main features:** Collaborative, non-algorithmic content curation tool. Users create "channels" containing any type of block (text, image, link, file) and can connect blocks across channels to form visual knowledge collections.
- **Advantages:** Highly flexible, open-ended structure suited to creative professionals; strong community of designers and researchers; no recommendation algorithm - editorial control belongs entirely to the user; public API available.
- **Disadvantages:** Deliberately niche and minimalist, limiting mass adoption; steep learning curve; limited discovery features; free tier has a block limit; UX is unfamiliar to general audiences.

Raindrop.io

- **Main features:** Cross-platform bookmark manager with collections, tags, highlights, and full-text search. Supports visual collection views and public sharing of bookmark collections.
- **Advantages:** Polished, feature-rich bookmark management; excellent browser extension; clean UI; good import/export support; available across all major platforms (web, mobile, desktop).
- **Disadvantages:** Primarily a personal productivity tool rather than a social discovery platform; weak community and follow features; lacks item metadata enrichment such as curator ratings; minimal trending discovery functionality.

Product Hunt

- **Main features:** Community-driven platform for discovering new technology products. Daily launches are upvoted by users; products accumulate reviews and discussions.
- **Advantages:** High-quality community of makers and early adopters; authoritative for product discovery; strong SEO; robust notification and discussion systems; strong brand recognition in the tech space.
- **Disadvantages:** Scope limited to technology products; collection and curation features are secondary to the launch feed; strong platform lock-in for makers; not suitable for general-purpose list creation; perceived as promotional by general users.

Letterboxd

- **Main features:** Social film diary and review platform. Users log films watched, write reviews, and create thematic list collections (e.g., "Best horror films of the 1970s"). Features a strong social follow graph and discovery via lists and reviews.

- **Advantages:** Excellent domain-specific UX for film enthusiasts; high-quality curator community; elegant list-building tools; strong personal diary/log integration; large and growing active user base.
- **Disadvantages:** Entirely domain-specific (films only); not generalisable to other content types; no public API for programmatic access; limited features for professional curators outside the film domain.

Listly

- **Main features:** Web-based list creation and curation tool primarily targeting content marketers and bloggers. Users create embeddable lists of links and resources, collaboratively curated, with social sharing options.
- **Advantages:** Embeddable widget model for external content sites; collaborative editing; good SEO for list pages; easy link import.
- **Disadvantages:** Outdated and cluttered UI; limited discovery and social features; primarily focused on B2B and marketing use cases; weak mobile experience; minimal active general community.

Notion (Public Databases)

- **Main features:** All-in-one productivity workspace supporting public-facing databases, linked resources, gallery views, and filtered list publications. Often used informally as a curated-list publication tool.
- **Advantages:** Extremely flexible; excellent formatting and media embedding; no-code database views; large existing user base; free tier available.
- **Disadvantages:** Not purpose-built for social discovery; no native follow or notification model for content subscriptions; slow public page load; no recommendation engine; no community or engagement features.

1.6. Project context

- application context
- technological context
- organisational context
- legal context

Indicate aspects that are important for the product. It should lead to the identification of constraints of the project and the chances for its development.

Application Context

The idea for ListItUp comes from looking at how people use the internet today and noticing what is missing. We describe the current social and market background that creates a real need for our web page.

- **Tired of Algorithms:** In today's digital landscape, there is a growing demand from users for high-quality, human-created information. People are getting tired of automated recommendations and confusing search results. There is a real demand for trustworthy, hand-picked information organized by actual humans.
- **The Creators:** Many internet users and influencers want platforms where they can share their specific knowledge and build an audience.
- **Market Opportunity:** While there are great list apps for specific hobbies (like Letterboxd for movies), there isn't a widely used, simple web platform where people can curate and share lists about absolutely anything. Our app fills that gap.

Technological Context

To make our app successful, we have to build it to fit seamlessly into the tools and devices people already use every day. This outlines the external technological landscape and the third-party ecosystems our project relies on to function.

- **No App Downloads:** Users expect things to just work in their browsers. We need to provide a smooth experience on both mobile and desktop without forcing anyone to download a native app.
- **Third-Party Ecosystem Reliance:** We can't build every single feature from scratch. We have to depend on external providers to make the platform work. We rely on external APIs for OAuth 2.0 social logins (Google, GitHub), Open Graph metadata for enriching links, cloud object storage for handling images, and transactional email services. This means our platform's success and uptime are partially tied to the service-level agreements and constraints of these external technologies.

Organisational Context

Our working environment is defined by the fact that we are students with strict deadlines and very limited resources. This defines the reality of our development environment, including our team structure and the strict external constraints we face.

- **Academic Deadlines:** We are operating as a cross-functional academic team of six members. Unlike a commercial startup, we are strictly bound by our university calendar, meaning we have a hard deadline and a limited 7-week window to deliver our Minimum Viable Product (MVP).
- **Budget and Resource Limitations:** Because this is a university project, we are operating with a budget of 0€, which forces us to prioritize open-source components, containerized solutions, and free-tier cloud hosting for our database and deployment. We are also developing this using our shared personal developer laptops rather than enterprise-provided equipment.
- **Development Methodology:** To manage our limited time, we are organizing our work in two-week sprints and using free collaboration tools like GitHub, Figma, and Slack to keep everyone on the same page.

Legal Context

Even though this is a university project, our app will live on the real internet, meaning we must respect real-world laws and regulations. This section covers those external laws, regulations, and societal rules that our software must comply with to operate legally.

- **Data Protection and Privacy:** Because we collect user emails and handle accounts, we are legally bound by European data protection laws, specifically GDPR / RODO. This requires us to guarantee external user rights, such as the ability to access, rectify, erase, and port their data.
- **Cookie Regulations:** We must comply with the ePrivacy Directive, meaning any cookies used beyond strictly necessary session ones require informed user consent.
- **User-Generated Content Liability:** We need clear Terms of Service. Users will keep the copyright to the lists they create, but we also need legal protection, so we aren't held responsible if a user posts a broken or malicious external link
- **Accessibility Standards:** We need to make sure our website is usable for as many people as possible, aiming to meet the WCAG 2.1 AA accessibility standards for our public pages.

Constraints of the project

Because we are building this for a university class, we have a few strict limitations we need to work around to make this project a reality.

- **Time:** We only have a 7-week window to finish and launch our Minimum Viable Product (MVP).
- **Budget:** We are students, so of course we have no money to spend on this project. This forces us to rely completely on open-source tools and the free tiers of cloud hosting services instead of using paid, premium solutions.
- **No Native Apps:** Building separate apps for iOS and Android is impossible with our current time and budget, so we must focus entirely on making the web version look and work perfectly on both mobile and desktop browsers.

- **Simple Recommendations:** We don't have the resources to build a complex machine-learning algorithm to recommend lists. For this MVP, our discovery feed will just rank lists based on basic engagement, like views, likes, and saves.

Chances for its development (Opportunities)

Even with our strict limitations and 0€ budget, there are several external factors in the current tech landscape that make it highly possible for us to successfully develop this project:

- **Massive Open-Source Ecosystem:** We can build an enterprise-level application for free because we have access to powerful open-source technologies. Tools like Spring Boot for the backend, MySQL for the database, and Docker for containerization are completely free to use.
- **Ready-to-Use External APIs:** Instead of wasting our short 7-week timeline trying to build complex features like secure login systems from scratch, we can integrate existing external tools. Relying on third-party APIs for things like Google and GitHub OAuth social logins saves us a massive amount of development time.
- **Free Automation and Deployment Tools:** External platforms like GitHub give us access to built-in CI/CD tools (GitHub Actions) for free. This means we can automate our testing and deployment processes without needing to pay for expensive enterprise software.
- **Our Team's Existing Knowledge:** We are using Java because all team members already have experience with it. Not having to learn a completely new programming language from scratch drastically improves our chances of finishing the MVP before the deadline.

Measurable Success Criteria (pilot)

- **Functional:** > 80% of pilot users can create and publish a list without staff assistance.
- **Reliability:** end-to-end list creation/publish success rate $\geq 98\%$ in QA tests.
- **Discovery:** average time-to-first-relevant-list < 10 seconds for a new visitor.
- **Engagement:** interaction rate (like, save, or comment) of at least 15% of active monthly users.

1.7. Technologies used in the project

Justify the choice of technology. Focus on each element and provide the required information

- technology name,
- description of the technology,
- justification,
- key responsibilities in the project, e.g., front-end development,
- link to the technology description.

Java (Backend Language)

- **Description:** General-purpose, statically typed, object-oriented programming language and runtime platform, widely used for building enterprise-grade server-side applications.
- **Justification:** Java offers strong static typing, mature tooling, and a vast ecosystem of libraries for building reliable backend services. Its strict type system reduces runtime errors, and the JVM provides consistent cross-platform execution. All team members had existing experience with Java, eliminating onboarding overhead within the limited project window.
- **Key responsibilities:** Implementation of all server-side business logic - resource management, reservation and loan lifecycle, user role handling, and API endpoints.
- **Reference:** java.com - docs.oracle.com/en/java

Spring Boot (Backend Framework)

- **Description:** Opinionated Java framework built on top of the Spring ecosystem that enables rapid development of production-ready web applications and REST APIs with minimal configuration.
- **Justification:** Spring Boot drastically reduces boilerplate configuration compared to plain Spring; it provides built-in support for REST controllers, dependency injection, JPA/Hibernate ORM, security, and transaction management - all essential for a resource management system. Its embedded server (Tomcat) simplifies deployment inside Docker containers.
- **Key responsibilities:** REST API layer, business logic services, authentication and authorisation (Spring Security), database access via Spring Data JPA, and application configuration management.
- **Reference:** spring.io/projects/spring-boot

HTML + CSS + JavaScript (Frontend)

- **Description:** Standard web technologies used to build the user interface: HTML5 for document structure, CSS3 for styling and responsive layout, and vanilla JavaScript for client-side interactivity and API communication.
- **Justification:** Given the scope of the MVP and the team's existing skills, a lightweight frontend built with plain web technologies offers the fastest path to a working interface without the overhead of a JavaScript framework. The backend serves Thymeleaf-rendered pages or static HTML/CSS/JS assets directly, keeping the stack simple and the deployment straightforward.
- **Key responsibilities:** All user-facing pages - resource browsing, reservation forms, loan management views, check-in/check-out interfaces, and admin panels.
- **Reference:** developer.mozilla.org/en-US/docs/Web

MySQL (Relational Database)

- **Description:** Open-source relational database management system that stores data in structured tables with full support for SQL, ACID transactions, and foreign key constraints.
- **Justification:** MySQL is a proven, lightweight relational database well-suited to the structured, transactional nature of resource management data (reservations, loans, users, policies). It pairs natively with Spring Data JPA and Hibernate, and its configuration is straightforward to manage inside a Docker container. The `mysql/conf` directory in the repository provides a custom configuration for the containerised instance.
- **Key responsibilities:** Persistent storage of all domain data: users, resources, reservations, loans, check events, holds, notifications, and policies.
- **Reference:** [mysql.com - dev.mysql.com/doc](https://dev.mysql.com/doc)

Nginx (Reverse Proxy & Web Server)

- **Description:** High-performance, open-source HTTP server and reverse proxy capable of serving static files, forwarding requests to application servers, and terminating TLS connections.
- **Justification:** Nginx acts as the single entry point for all incoming traffic, forwarding requests to the Spring Boot application while handling HTTPS termination via the `setup-https.sh` script present in the repository. This separation improves security, enables efficient static asset serving, and makes it straightforward to add rate limiting or load balancing in future phases.
- **Key responsibilities:** Reverse proxy for the Spring Boot API, HTTPS/TLS termination, and static asset delivery.
- **Reference:** nginx.org/en/docs

Docker + Docker Compose (Containerisation & Orchestration)

- **Description:** Docker is a containerisation platform that packages applications and their dependencies into portable, isolated containers. Docker Compose is a tool for defining and running multi-container applications via a single YAML configuration file.
- **Justification:** The `docker-compose.yml` at the root of the repository orchestrates all services - the Spring Boot application, MySQL database, and Nginx reverse proxy - as a unified stack. This guarantees environment consistency between development and production, simplifies onboarding (single command startup), and makes the deployment reproducible with no manual environment configuration.
- **Key responsibilities:** Containerisation of all system services, local development environment setup, and production deployment orchestration.
- **Reference:** [docs.docker.com - docs.docker.com/compose](https://docs.docker.com/compose)

GitHub Actions (CI/CD)

- **Description:** Integrated CI/CD automation platform built into GitHub that triggers configurable pipelines on repository events such as push or pull request.
- **Justification:** As the codebase is hosted on GitHub, GitHub Actions provides native, zero-configuration CI/CD integration. It enables automated build and test execution on every pull request and can trigger Docker image builds and deployment to the production host,

keeping the pipeline auditable and the deployment process repeatable without additional tooling costs.

- **Key responsibilities:** Automated build verification, test execution, Docker image build, and deployment promotion to the production environment.
- **Reference:** docs.github.com/actions